

AI-Powered Unified Vulnerability Intelligence Platform

Multi-Layer Security Intelligence | AI Security Agent | OAuth-Based Repository Access

WHAT TYPES OF VULNERABILITIES ARE WE FOCUSING ON?

We focus on 3 layers.

Layer	Tool	What We Detect
Application	Semgrep	Code logic flaws
Dependency	Trivy	Library CVEs
Container	Trivy	OS & Docker issues

Application (8 types)

- SQL Injection
- XSS
- Command Injection
- Path Traversal
- SSRF
- Hardcoded Secrets
- Broken Access Control
- Weak Cryptography

Dependency (4 types)

- Known CVEs
- High CVSS

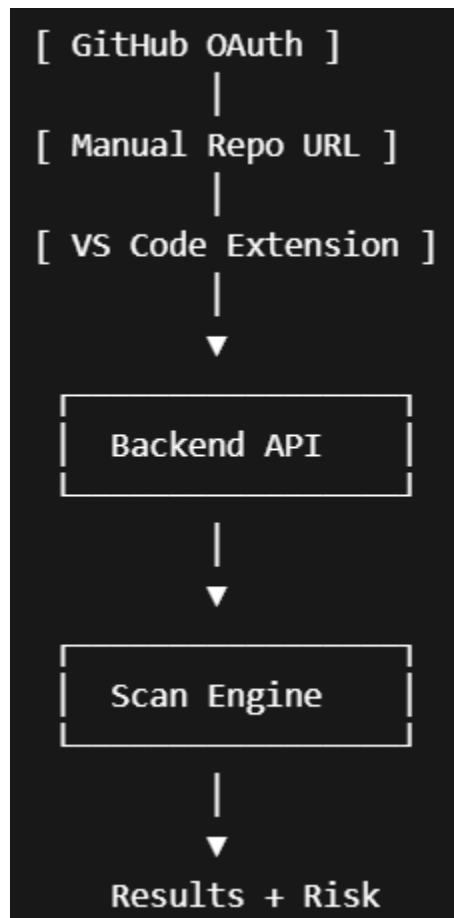
- Transitive vulnerabilities
- Outdated packages

Container (4 types)

- Vulnerable base image
- Root user
- Exposed ports
- Secrets in layers

Total scope: ~16 structured vulnerability categories.

3 ENTRY MODES:



- ◆ **Entry Point A: GitHub OAuth**
- ◆ **Entry Point B: Public/Private Repo URL Input**
- ◆ **Entry Point C: VS Code Extension**

◆ **1 FEATURE 1: GitHub Login + Repo Selection**

This is cleanest approach.

Flow:

User clicks login

- GitHub OAuth
- Backend receives token
- Fetch repos via GitHub API
- User selects repo
- Backend clones repo using token
- Scan starts

Best for:

- Private repos
- Clean UX
- Authenticated users
- Scan history tracking

◆ **2 FEATURE 2: Manual Repository URL Input**

Now this is important.

User pastes:

`https://github.com/username/project`

We handle 2 cases:

Case A: Public Repo

We can clone directly.

`git clone https://github.com/username/project.git`

No authentication required.

Case B: Private Repo

We cannot clone unless:

- User logged in via GitHub OAuth
- Or user provides access token

So logic becomes:

IF user logged in → use stored token

ELSE → allow only public repo scanning

That is safe and clean.

SECURITY CONSIDERATIONS FOR MANUAL URL

This is where teams make mistakes.

You must:

- Validate URL format
- Prevent command injection
- Restrict cloning depth
- Set repo size limit
- Timeout scan after X minutes

Never blindly run:

```
git clone ${userInput}
```

Sanitize first.

◆ 3 FEATURE 3: VS CODE EXTENSION (Dynamic Analysis Trigger)

Now this part needs clarification.

Right now your system is doing:

SAST + SCA → static analysis

Dynamic analysis means:

- Runtime testing
- Executing code
- Fuzzing
- Observing behavior

Are you planning real dynamic analysis?

Or:

Extension triggers scan on current local project?

These are very different.

◆ Real Dynamic Analysis (Advanced)

That would require:

- Running app in sandbox

- Sending test payloads
- Observing runtime behavior
- Possibly DAST tools

This is complex and probably too heavy for 4 months.

◆ **Recommended Version (Realistic)**

VS Code extension:

- Authenticates user
- Sends current project folder to backend
- Backend scans like usual
- Returns vulnerabilities
- Extension highlights vulnerable lines
- AI provides fix suggestions

This is still static analysis, but locally triggered.

And it is completely realistic.

◆ **HOW VS CODE EXTENSION SHOULD WORK**

Flow:

Developer opens project

→ Extension activated

→ User clicks "Scan Project"

→ Extension compresses project

→ Sends to backend `/scan-local`

- Backend runs same scan engine
- Returns JSON results
- Extension highlights lines
- Hover shows AI explanation

This is elegant and feasible.

◆ **IMPORTANT ARCHITECTURE DECISION**

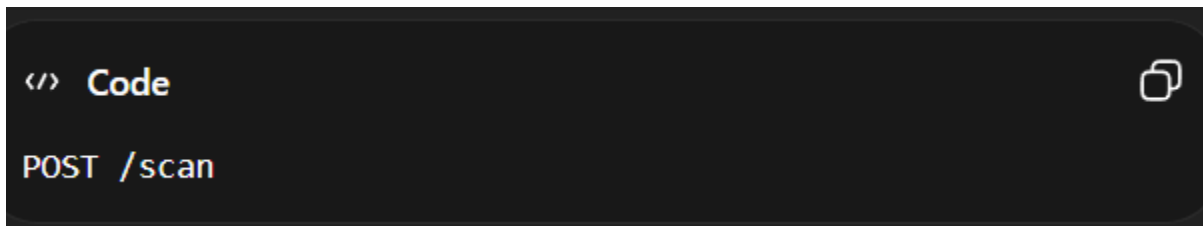
We must ensure:

There is **ONLY ONE** scan engine.

Whether:

- OAuth
- Manual URL
- VS Code

They all call:

A screenshot of a code editor with a dark theme. In the top left corner, there is a code icon (two overlapping arrows) followed by the word "Code" in a light blue font. In the top right corner, there is a copy icon (two overlapping squares). The main area of the editor shows a REST client request: "POST /scan" in a light blue font. The text is on a single line, and the background is a solid dark color.

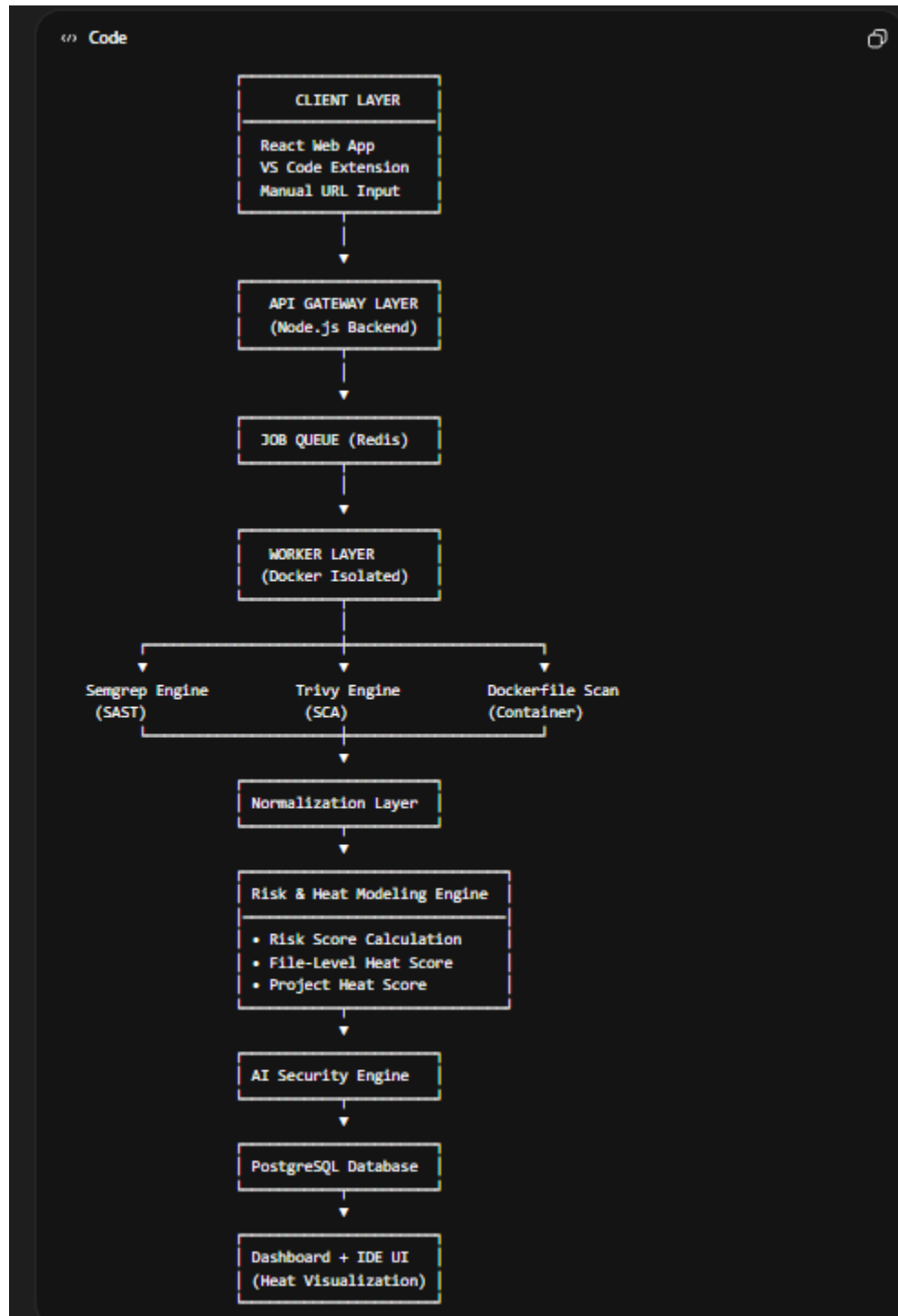
```
</> Code
```

```
POST /scan
```

With different parameters.

That keeps architecture clean.

— ARCHITECTURE OVERVIEW:



Our system follows a layered, centralized security intelligence architecture. Multiple clients including the web dashboard and VS Code extension, communicate with a unified backend API. Scan requests are queued using Redis and executed inside isolated Docker containers using SAST and SCA engines. Results are normalized, risk-modeled and enhanced with AI-generated remediation guidance. All data is stored in a relational database to enable historical tracking and trend analysis.

ARCHITECTURE WITH HEAT MAP INTEGRATION

LAYER 1 — CLIENT LAYER

Components:

- React Web App
- VS Code Extension
- Manual URL Scanner UI

Responsibilities:

- User interaction
- Sending scan requests
- Displaying vulnerabilities
- Showing AI suggestions
- Showing risk trends
- **Displaying heat map (file-level & line-level)**
- **Color-coded severity visualization**

This layer does NOT:

- Run scanning tools
- Calculate risk
- Detect vulnerabilities
- Compute heat scores

It only requests services and visualizes results.

● LAYER 2 — API GATEWAY (NODE.JS BACKEND)

This is the brain controller.

Responsibilities:

- Authentication (GitHub OAuth + JWT)
- Authorization
- Validate repo URLs
- Accept file uploads (VS Code)
- Create scan jobs
- Store scan metadata
- Return results (including risk + heat data)

Important:

This layer does NOT directly run heavy scanning.
It delegates to workers.

● LAYER 3 — JOB QUEUE (REDIS)

Why Redis?

Because scanning:

- Takes time
- Is CPU heavy
- Cannot block HTTP requests

Flow:

Client → API → Redis → Worker picks job

Benefits:

- Prevents overload
- Handles concurrency
- Scales horizontally

If 50 users trigger scan:
Queue manages execution order.

LAYER 4 — WORKER LAYER (DOCKER ISOLATED)

Critical for security.

For each scan:

- Create isolated Docker container
- Mount project inside
- Run scanning tools
- Collect results
- Destroy container

Why isolation?

User code is untrusted.
Never run untrusted code on the host machine.

LAYER 5 — SECURITY ENGINES

SAST Engine (Semgrep)

Detects:

- SQL Injection
- XSS
- SSRF
- Command Injection
- Hardcoded secrets

SCA Engine (Trivy)

Detects:

- CVEs in dependencies
- High CVSS libraries

Container Scan (Trivy Docker)

Detects:

- Base image vulnerabilities
- Root user
- Exposed ports

Each produces a different JSON output.

LAYER 6 — NORMALIZATION LAYER

Converts all scanner outputs into unified schema:

```
{  
  scan_id,  
  layer,  
  vulnerability_type,  
  severity,  
  cvss,  
  file,  
  line,  
  description,  
  status  
}
```

Without normalization → inconsistent system.

With normalization → structured intelligence.

● LAYER 7 — RISK & HEAT MODELING ENGINE ✓

This is your intelligence core.

Input:

All normalized vulnerabilities.

Process:

1. Group by severity
2. Apply layer weights
3. Apply CVSS scores
4. Add persistence factor
5. Add exposure multiplier
6. Compute overall Risk Score (0–100)
7. **Compute file-level heat score**
8. **Compute project-level heat score**

Risk Formula:

Risk Score =

$\Sigma (\text{CVSS} \times \text{LayerWeight})$

+ CriticalBonus

+ PersistenceFactor

Heat Computation (New Addition):

Vulnerability Heat = $\text{CVSS} / 10$

File Heat =

$$\Sigma (\text{CVSS of file}) / (10 \times \text{total_vulnerabilities_in_file})$$

Project Heat =

Risk Score / 100

Output:

- Risk Score (0–100)
- Risk Category (Low / Medium / High / Critical)
- File-level heat score
- Project-level heat score

This enables visual heat map representation.

LAYER 8 — AI SECURITY ENGINE

AI does NOT scan.

AI receives:

- Code snippet
- Vulnerability type
- Severity
- Context

AI returns:

- Explanation
 - Why dangerous
 - Secure fix
 - Best practice
-

● LAYER 9 — DATABASE LAYER (POSTGRESQL)

Tables:

- Users
- Projects
- Scans
- Vulnerabilities
- Scan_Events
- AI_Responses
- **(Optional) File_Heat_Scores**

Relationships:

User → Projects

Project → Scans

Scan → Vulnerabilities

Scan → Events

Enables:

- History tracking
- Risk trends
- Alerting
- **Heat map tracking per scan**

You may either:

- Store heat scores
- OR
- Recalculate dynamically from vulnerabilities

Both are valid.

DATA FLOW (END-TO-END)

◆ TRACE¹ — VS Code Extension + Heat Map

1. Developer opens project in VS Code
2. Developer clicks **"Scan Project"**
3. Extension collects workspace files
4. Extension excludes:
5. node_modules
6. .git
7. build folders
8. Extension zips project
9. Extension sends POST `/scan-local`
10. Extension includes Authorization token
11. API validates token
12. API stores scan record in database
13. API pushes job to Redis
14. Worker pulls job
15. Worker creates isolated Docker container
16. Worker unzips project inside container
17. Runs Semgrep
18. Runs Trivy
19. Runs Dockerfile scan (if present)
20. Collect JSON outputs
21. Normalize results
22. Compute risk score
23. Compute heat score per file
24. Compute project heat score
25. Store vulnerabilities + heat data in DB
26. API notifies client
27. Extension receives:
28. Vulnerabilities
29. File heat scores
30. Project risk score

31. Extension applies:
 32. Line-level heat coloring
 33. File-level heat badges
 34. Severity color coding
 35. Developer visually sees hot zones
 36. Developer clicks vulnerability
 37. Extension requests AI explanation
 38. AI explanation returned
-

◆ **TRACE²** — GitHub Authorization + Heat Map

1. User clicks **"Login with GitHub"**
2. User authorizes application
3. Backend receives authorization code
4. Backend exchanges code for GitHub access token
5. Backend stores token securely
6. Backend generates JWT
7. Frontend receives JWT
8. User requests repository list
9. Backend fetches repos from GitHub
10. Frontend displays repositories
11. User selects repository
12. Frontend sends POST **/scan-repo**
13. API validates JWT
14. API creates scan record
15. API pushes job to Redis
16. Worker pulls job
17. Worker creates Docker container
18. Worker clones repository using GitHub token
19. Runs Semgrep
20. Runs Trivy
21. Runs Dockerfile scan (if present)
22. Collect JSON outputs
23. Normalize results
24. Compute risk score

25. Compute heat score per file
 26. Compute project heat score
 27. Store vulnerabilities + heat data in DB
 28. API notifies client
 29. Dashboard receives:
 30. Vulnerabilities
 31. File heat scores
 32. Project risk score
 33. Dashboard displays:
 34. Heat-colored file tree
 35. Risk meter
 36. Vulnerability table
 37. User clicks vulnerability
 38. Frontend requests AI explanation
 39. AI explanation returned
-

◆ **TRACE³** — Manual Repository URL + Heat Map

1. User pastes repository URL
2. User clicks "**Scan**"
3. Frontend sends POST `/scan-url`
4. API validates repository URL
5. API checks public/private access
6. API creates scan record
7. API pushes job to Redis
8. Worker pulls job
9. Worker creates Docker container
10. Worker clones repository using HTTPS
11. Runs Semgrep
12. Runs Trivy
13. Runs Dockerfile scan (if present)
14. Collect JSON outputs

15. Normalize results
 16. Compute risk score
 17. Compute heat score per file
 18. Compute project heat score
 19. Store vulnerabilities + heat data in DB
 20. API notifies client
 21. Dashboard receives:
 22. Vulnerabilities
 23. File heat scores
 24. Project risk score
 25. Dashboard displays:
 26. Heat-colored file tree
 27. Risk ranking
 28. Vulnerability table
 29. User clicks vulnerability
 30. Frontend requests AI explanation
 31. AI explanation returned
-

◆ **KEY OBSERVATION (Very Important)**

In all three flows:

After this step:

Worker clones/unzips project

Everything becomes identical:

- Same scan engine
- Same normalization

- Same risk engine
- Same heat map logic
- Same AI explanation

Only the **input source differs**.

◆ Unified Architecture Summary

All three modes follow:

Input → Scan Engine → Normalize → Risk → Heat Map → Store → Visualize → AI

That's your entire system in one sentence.